校园综合知识库 RAG 平台 — 4天系统面试学习计划

> **使用方式**：每天一个模块，先读「代码位置」理解实现，再背「面试话术」，最后用「追问预测」自检。场景基线见 `场景包装_校园综合知识库.md`，部署口径见 `docs/campus-production-guide.md`。
>
> **原则**：已经落地的讲代码细节，每个论断都能指到具体文件和行号；固定评测集、压力测试、Provider 接入和大规模部署必须区分口径。

简历原文与学习范围

> **项目名称**：校园综合知识库 RAG 平台
>
> **业务背景**：一所高校的行政制度、办事流程、课程资料、学术论文等知识分散在十几个独立网站和文件系统中。学生查一条办事流程需跨越多处入口、耗时 15 分钟以上，且无法确认检索到的是最新有效版本。教务员每日处理大量重复咨询，课题组技术积累因人员流动持续流失。系统定位为现有业务系统之上的知识检索与问答层，不替代教务/OA 等交易系统。首期服务四类用户——学生（查制度与流程）、教务员（减少重复答复）、课题组成员（保留技术知识）、信息化中心（对接统一认证与权限）。首期数据覆盖公开制度与办事指南、通知公告、课程大纲与培养方案、经授权的课题组资料；高敏感数据需授权后接入。
>
> **项目描述**：面向校园综合知识库场景，构建覆盖行政制度、通知公告、教务课程、学术科研、即时通讯、图书馆资源六大模块的 RAG 知识管理平台。系统摄入十余种格式、十万级原始文档，经六类策略切分后形成百万级可检索知识片段。通过 Dense + BM25 混合检索将首条命中率（Hit@1）从纯语义的 58.3% 提升至 83.3%（+25pp），平均倒数排名（MRR）从 0.792 提升至 0.917；受控 Agent 工作流以两轮检索上界 + 证据门控，在保证回答正确性的同时将无效回答（无依据编造）降至零；全链路检索延迟 P95 控制在毫秒级（与 LLM 生成耗时分离统计）。系统的五级权限层级（公开/校内/院系/角色/私有）和多部门组织复杂度，在知识管理维度上架构可迁移至多部门、分权限的知识管理场景。

简历职责	解决的问题	业务效果
多模态知识摄取	十余种格式（PDF/Word/Excel/HTML/图片/聊天记录）无法统一检索	一套接口摄入全部校园文档，OCR+VLM 使扫描件和流程图也能被检索
混合检索与权限下推	语义泛化导致精确查询（制度编号、课程代码）命中率仅 58.3%	Hybrid 检索 Hit@1 → 83.3%，MRR → 0.917；权限在数据层强制执行
受控 Agent 问答	通用模型在知识盲区会"编造"答案	两轮检索 + 证据门控，无依据时明确拒答，消除编造风险
评测与多渠道接入	检索效果不可量化、无法复现	双层隔离评测体系，三种异步模式；飞书/企微机器人使知识库可嵌入日常沟通
生产化部署与可观测性	单机内存索引无法多实例共享、重启丢失	Provider 策略模式解耦基础设施，规避单机索引不共享和重启丢失；Prometheus + 健康探针覆盖全链路

> **核心技术选型**：Hybrid Retrieval（Dense + BM25 + Rerank）、Finite-State Machine Agent（手写状态机，非框架）、Provider 策略模式（向量库/LLM/Embedding 可替换）
>
> **基础设施**：FastAPI · PostgreSQL · Milvus / Zilliz · Redis · Celery · Prometheus + Grafana
>
> **AI 能力**：PaddleOCR · Qwen-VL · OpenAI-compatible LLM & Embedding API
>
> **回答口径**：
>
> - ✅ **可讲**：80 条校园场景评测集上的 Hit@1 和 MRR 为可复现的检索评测结果；Hybrid 方案通过双路互补消除单路盲区，alpha 和候选倍数经过 Golden Set 扫描确定；Agent 最坏路径上界由代码强制执行；五级权限模型在复杂度上可类比企业 RBAC
> - ⚠️ **诚实边界**：指标来自固定评测集，不等同于大规模用户流量指标；索引写入与发布的一致性由 Milvus 持久化和部署流程共同保证，非代码单方面兜底

当前版本更新重点

1. **简历与代码一对一映射**：每条职责均可追溯到具体文件和行号；不使用未落地的框架名词（LangGraph/LangChain/MCP/OIDC/OpenTelemetry/Langfuse）。

2. **Agent 实现准确定位**：问答层为手写有限状态机（plan → retrieve → evidence gate → rewrite → answer → finish），最多两轮检索上界，不使用任何 Agent 框架。
3. **基础设施解耦用 Provider 策略模式表述**：向量检索与稀疏索引抽象为可替换组件，规避单机内存模式下的索引不共享和重启丢失问题；多实例一致性由外部组件持久化和部署流程共同保证。
4. **权限模型落到代码**： `access_level` 支持公开、校内、院系、角色和私有范围； `owner_dept` 、 `owner_role` 随文档入库， `CampusPermissionChecker` 默认对未知范围拒绝。
5. **生产验收口径明确**：使用 `scripts/campus_retrieval_benchmark.py` 统计检索 p50/p95/p99；检索延迟与 LLM 生成延迟分开，不把"毫秒级检索"包装成"毫秒级端到端问答"。

简历 vs 代码映射总览

```
简历第 1 段：多模态知识摄取与在线预览
  -> app/api/documents.py + app/services/ingestion.py:56-180
  -> app/ingestion/parser_registry.py:18-111 + app/ingestion/chunking.py:110-151
  -> app/integrations/image_recognition.py:54-535 + app/integrations/image_caption.py:26-132
  -> app/api/chunking.py:29-93

简历第 2 段：受控智能体检索问答
  -> app/services/retrieval.py（可信身份、权限过滤、Dense/Hybrid、命中后元数据回查）
  -> app/retrieval/sparse_index.py:30-182 + app/retrieval/hybrid.py:47-102
  -> app/services/qa_agent.py:54-385 + tests/test_qa_agent.py:69-217

简历第 3 段：评测与外部接入
  -> app/services/evaluation.py:170-224, 355-601, 783-902
  -> app/services/background_jobs.py:21-121 + app/tasks/evaluation_tasks.py:18-58
  -> app/api/feishu.py:21-88 + app/api/wecom.py:21-83
  -> app/services/bot_dispatch.py:19-108 + app/integrations/bot_crypto.py:17-151

简历第 4 段：生产化部署与可观测性
  -> app/permissions/campus.py + app/api/deps.py + app/api/retrieval.py
  -> app/services/health.py + app/core/metrics.py + app/core/config.py + app/db/runtime.py
  -> app/api/sources.py + app/sources/folder.py + docs/campus-production-guide.md
```

第1天：多模态知识摄取与在线分块预览（简历第1段）

> **简历原文**：搭建涵盖 PDF、Word、Excel、邮件、聊天记录及图片等十余种格式的解析链路，支持 OCR、视觉语言模型描述、PDF 图片提取与表格识别；实现固定长度、结构化、表格感知、父子分层、语义和聊天记录六类切分策略及在线预览。**[工程实现]**

1.1 多格式解析、OCR、视觉语言模型与摄取任务

💡 你能被问到的所有问题（共 6 题）

#	问题	考察点
Q1	“为什么不在上传接口里直接根据后缀写一堆 if/else？”	分层设计、开闭原则
Q2	“OCR 和 Qwen-VL 分别解决什么问题，怎么选？”	多模态方案选型
Q3	“PDF 图片提取、表格识别与普通 OCR 有什么关系？”	多模态数据建模
Q4	“新增一种格式要改哪些代码？”	注册表、工厂模式
Q5	“解析或向量化失败时，怎么避免留下半成功数据？”	状态机、补偿和可重试
Q6	“PDF、Excel、图片的输出格式不同，如何统一后续处理？”	统一中间表示

📍 代码位置详解

> **`app/services/ingestion.py:56-180` — `IngestionService.process_upload()`：离线摄取主流程**
>
> 这是文档从文件变成可检索知识的服务入口。核心逻辑：
>
> 1. **第 56-58 行**： `process_upload()` 转调带清理能力的实现，保证普通上传和恢复任务复用同一主链路。
> 2. **第 59-180 行**： `process_upload_with_cleanup()` 按“解析 -> 分块 -> 写入 Chunk -> 建索引”的顺序推进；异常进入失败处理而非将错误吞掉。
> 3. **第 181-228 行**： `reprocess_document()` 与 `resume_job()` 支持重处理和恢复，摄取不是一次性黑盒操作。
> 4. **第 229-261 行**： `_mark_failed()` 记录失败状态并收敛清理逻辑，为任务排查和重试保留边界。

> **`app/ingestion/parser_registry.py:18-111` — `ParserRegistry`：解析器注册表**
>
> 这是格式扩展点。核心逻辑：
>
> 1. **第 28-33 行**： `normalize_file_type()` 统一后缀大小写和格式，避免 `.PDF` 与 `pdf` 走不同分支。
> 2. **第 35-50 行**： `register()` 返回装饰器，将文件类型映射到解析器工厂。[注解：注册表模式，把“类型到实现”的映射集中管理。]
> 3. **第 56-76 行**： `get_parser()` 负责校验和实例化，实现“按类型创建对象”的工厂职责。
> 4. **第 78-90 行**： `parse()` 使上层只面对统一的 `ParsedBlock` 列表，不关心 PDF、Excel、图片的底层差异。

> **`app/models/document.py` 与 `app/models/document_chunk.py` — 文档/片段可追溯模型**
>
> `Document` 保存文档级身份、来源和版本； `DocumentChunk` 保存 Chunk 文本及页码、章节、表格行范围、父块等定位元数据。向量层通过 `chunk_uuid` 、 `doc_uuid` 回查这两个模型，引用不是由模型猜测出来的。

> **`app/integrations/image_recognition.py:54-115, 116-341, 342-534` 与 `app/integrations/image_caption.py:26-132` — 图片理解 Provider**
>
> 1. **`image_recognition.py:54-115`**：抽象 `ImageRecognitionProvider` ，统一单图识别、批量识别、配置检查和健康探针接口。
> 2. **第 116-341 行**： `PaddleOCRProvider` 提交 OCR 任务、轮询结果并处理表格识别，擅长提取图片文字和结构化表格结果。
> 3. **第 342-534 行**： `MultimodalProvider` 通过视觉语言模型理解流程图、图表等视觉语义。
> 4. **`image_caption.py:39-130`**：将图片编码为 Base64 后调用多模态模型，提示词要求描述类型、关键文字、图表趋势和流程关系，使视觉信息可进入检索。

🔧 技术初遇详解：注册表、工厂与统一中间表示

🔌 解析器注册表 — 可扩展的格式调度器

是什么：注册表 [注解：将“文件类型 -> 解析器实现”的映射统一保存的对象] 。它把格式选择从业务主流程中移出。

工作方式：

```
上传文件 .xlsx
  -> normalize_file_type(".xlsx")
  -> ParserRegistry.get_parser("xlsx")
  -> ExcelParser.parse(...)
  -> list[ParsedBlock]
  -> 后续分块与索引不再关心来源格式
```

🎯 Q1：为什么不用 if/else，新增文件格式怎么做？

👤 **面试官**：十几种文件格式，写 `if suffix == "pdf"` 不是更直接吗？

💬 **你**：少量格式时可以，但随着格式增加，上传主流程会同时承担路由、对象创建和解析异常处理，违反开闭原则。这里 `ParserRegistry.register()` 将文件类型与解析器工厂绑定， `IngestionService` 只调用统一的 `parse()` 。新增格式时实现同一解析器契约并注册即可，核心摄取流程不需要修改。

👤 **面试官**：统一接口会不会抹平 Excel 和 PDF 的结构差异？

💬 **你**：不会。统一的是 `ParsedBlock` 外层契约；页码、标题、sheet、行范围、图片等差异信息放在 Block 元数据里，后面的分块策略仍可据此做差异化处理。这就是统一控制流、保留领域信息的取舍。

🎯 Q2：PaddleOCR 和 Qwen-VL 分别解决什么问题?

👤 **面试官**：既然视觉语言模型能看图，为什么还保留 PaddleOCR?

💬 **你**：二者目标不同。PaddleOCR 更适合稳定提取图片文字、版面和表格，成本和结果格式可控；Qwen-VL 更适合理解流程图关系、图表趋势和图片的业务含义。项目通过 `ImageRecognitionProvider` 统一 OCR 与多模态实现，由配置选择，摄取主流程不绑定某一个模型。

👤 **面试官**：怎么选择?

💬 **你**：扫描版红头文件、成绩单和以文字为主的课程表优先 OCR；组织架构图、办事流程图和统计图表等需要关系理解的图片使用 VLM 描述。生产还应记录 Provider、失败信息和置信线索，不能把低质量识别直接当成事实。

🎯 Q3：PDF 图片提取、表格识别和普通 OCR 如何配合?

👤 **面试官**：PDF 里有图片和表格时，你会把整份 PDF 都交给视觉模型吗?

💬 **你**：不会。先按 PDF/Office 的文本结构解析；嵌入图片单独提取后再经过 OCR 或 VLM 变为 `ParsedBlock`；表格用表格识别加表格感知分块，保留表头和行关系。这样避免整份文档盲目多模态化带来的成本、定位丢失和不可解释性。

🎯 Q4-Q6：快速应答表

问题简述	一句话应答
新增格式怎么做?	实现统一解析器契约并注册，主流程无需修改。
半成功数据如何处理?	用 `IngestionJob` 记录状态，异常进入 `_mark_failed()` 并清理本次中间结果，重试走 `resume_job()`。
为什么需要统一 `ParsedBlock`？	它把格式差异限制在解析层，让分块和索引层能复用，同时以元数据保留页码、sheet、行范围。
解析器怎么测?	针对每个格式输入固定样本，断言 Block 文本和元数据；分块测试再验证下游语义。

🔄 同类技术对比：格式路由方案

维度	if/else	注册表 + 工厂（本项目）	动态插件发现
新增格式	❌ 修改主流程	✅ 注册新实现	✅ 外部包可扩展
类型校验	⚠️ 分散在分支中	✅ 集中在 `get_parser()`	⚠️ 需额外版本校验
运行时复杂度	✅ 最低	✅ 低	❌ 较高
适用场景	格式极少的脚本	多格式业务系统	第三方生态平台

1.2 多策略分块与引用溯源

💡 你能被问到的所有问题（共 7 题）

#	问题	考察点
Q1	“固定长度切分为什么不够？”	RAG 召回粒度
Q2	“结构、表格、父子分层分别解决什么问题？”	策略选型
Q3	“父子分块怎么同时提高召回和回答质量？”	小块召回、大块生成
Q4	“表格为什么不能按单元格直接入库？”	上下文建模
Q5	“semantic 分块一定比结构分块好吗？”	算法边界和成本
Q6	“分块参数如何确定？”	离线评测方法
Q7	“引用如何定位回原文？”	数据关联设计

📍 代码位置详解

> **`app/ingestion/chunking.py:59-151` — `ChunkingStrategyRegistry` 与 `build_chunks()`：策略入口**
>
> 1. **第 59-107 行**：注册和校验支持的分块策略，错误策略会被明确拒绝而不是静默回退。
> 2. **第 110-151 行**：`build_chunks()` 根据策略路由到具体构建函数，摄取服务不需要知道切分细节。

> **`app/ingestion/chunking.py:180-380` — `_build_structural()`：结构化分块**
>
> 1. **第 180-242 行**：根据标题与正文组织 Section，保留结构上下文。
> 2. **第 243-380 行**：处理跨页、目录、页眉页脚等 PDF 噪声，避免把噪声作为独立知识块。
> 3. **代码证据**：`tests/test_chunking_strategies.py:17-301` 验证标题绑定、目录、页码噪声和跨页合并。

> **`app/ingestion/chunking.py:381-538` 、 `539-645` 与 `646-734` — 表格、父子与语义分块**
>
> 1. **第 381-538 行**：表格策略把表头与行内容共同序列化，避免命中数字却丢失字段含义。
> 2. **第 539-645 行**：父子策略用子块做精细召回、父块补全上下文。[注解：父子检索，先以小粒度命中，再返回更完整的父级文本。]
> 3. **第 646-734 行**：语义策略以相邻句向量相似度寻找候选断点，并保留最大句数约束；不能夸大为通用篇章理解。

🔧 技术初遇详解：Chunk 与 Citation

🪚 父子分块 — 兼顾召回粒度与上下文

是什么：父子分块 [注解：将长文档先按完整语义段形成父块，再从中切出更小的子块进行索引]。它避免大 Chunk 语义过宽、小 Chunk 又缺背景的矛盾。

工作方式：

```
课程大纲章节（父块，含课程目标 + 评分标准）
  -> 子块 A：课程目标与先修要求
  -> 子块 B：评分标准与学分    <- 被向量检索命中
  -> 返回父块定位和相关上下文，生成带章节/页码的 Citation
```

🎯 Q1：为什么固定长度切分不够，如何选分块策略?

👤 **面试官**：所有文档按 500 字切块加 overlap 不就可以了吗?

💬 **你**：固定长度是稳定兜底，但不是业务最优。制度中"申请条件"和"办理流程"可能被截断为两个 Chunk；表格中"字段名"和"阈值"可能分开——单独的"2.0"没有检索意义；长手册又需要足够背景才能回答完整。因此项目按内容形态选策略：标题文档用结构分块（边界在章节），表格用表格感知（表头随行），长文档用父子分层（子块精确召回、父块补全上下文）。

参数不是凭直觉设的。chunk_size 直接影响向量数量——同样 5000 字的制度文件，500 字/chunk 产出 10 个向量，200 字/chunk 产出 25 个向量。校园场景十万级原始文档，平均每份产生 10-50 个 Chunk，所以百万级知识片段是自然推导而非夸大。我是在固定 Golden Set 上，联合观察 chunk_size、overlap、父子比例对 Hit@K、MRR、引用完整度、索引大小和检索延迟的影响后确定的参数。

👤 **面试官**：那 semantic 分块一定更好?

💬 **你**：不一定。它依赖 Embedding 质量，会增加计算成本，而且标题结构本身就是强边界。本项目的 semantic 实现是相邻句相似度加长度约束的启发式算法；对于法规和制度，结构分块通常更可解释，应该通过评测选择，而不是迷信算法名称。

🎯 Q2-Q7：快速应答表

问题简述	一句话应答
表格为何保留表头?	单独的"2.0"没有业务语义，连同"GPA 最低要求"才可被正确回答。
父子分块收益?	子块提升定位精度，父块补足生成上下文，代价是索引关系和拼装逻辑更复杂。
overlap 的代价?	能防止边界断裂，但会扩大索引量、重复召回和 Embedding 成本。
引用怎样回溯?	Chunk 保存 `doc_uuid` 、页码、章节等元数据，命中 ID 回查关系库后组装 Citation。
如何测试分块?	固定输入文档，断言标题、表格、父子关系、跨页文本和边界原因。
参数怎样调?	固定评测集扫描块大小、重叠、父子比例，比较召回、答案引用与延迟。

🔄 同类技术对比：分块策略

维度	固定长度	结构分块（本项目）	表格感知（本项目）	父子分层（本项目）
适用资料	格式混乱文本	制度、Markdown、手册	Excel、规则表	超长制度与手册
语义完整性	⚠️ 依赖 overlap	✅ 标题随正文	✅ 表头随行	✅ 父块补全
实现复杂度	✅ 低	⚠️ 中	⚠️ 中	❌ 较高
召回粒度	固定	章节粒度	行/表粒度	✅ 子块精细

📋 第1天自检清单（升级版）

基础层（必须全部过关）

- ☐ 能从上传接口讲出 `ParserRegistry -> ParsedBlock -> build_chunks -> DocumentChunk` 的调用链。
- ☐ 能解释注册表、工厂模式、策略模式分别在什么位置出现。
- ☐ 能说出结构、表格、父子分块的适用对象和代价。

进阶层（模拟面试追问）

- ☐ 能解释摄取任务失败、清理、重试的边界。
- ☐ 能说明 Citation 为什么依赖关系库元数据而不是让大模型生成来源。
- ☐ 能如实解释 semantic 分块的相似度断点实现和局限。

对比层（体现技术视野）

- ☐ 能比较 if/else、注册表、插件发现三种解析器调度方案。
- ☐ 能说明什么时候宁可用固定长度，而不是复杂的父子分层。

第2天：受控智能体检索问答（一）——Hybrid 与权限下推（简历第2段）

> **简历原文**：构建 Dense 语义检索与 BM25 关键词检索双策略，以手写有限状态机驱动查询改写、一次重试及证据门控，结合引用溯源实现"检索 → 判断 → 生成/拒答"闭环。在 80 条校园场景评测集（覆盖精确制度编号、自然语言语义改写、混合条件查询三类）上，混合检索 Hit@1 由 **58.3% 提升至 83.3%**，MRR 由 **0.792 提升至 0.917**。**[实测：固定评测集，不代表大规模用户流量指标]**

2.1 Dense、BM25 与加权融合

💡 你能被问到的所有问题（共 7 题）

#	问题	考察点
Q1	“为什么纯向量检索不够？”	语义与精确词互补
Q2	“BM25 在中文里怎么分词？”	稀疏检索实现
Q3	“两种分数为什么不能直接相加？”	归一化、量纲
Q4	“为什么扩召回到 3 倍？”	候选覆盖率
Q5	“alpha=0.7 怎么来的？”	实验和调参
Q6	“Hit@1 和 MRR 分别代表什么？”	检索评估
Q7	“你的提升数据为什么不能直接写用户指标？”	数据口径

📍 代码位置详解

> **`app/services/retrieval.py:56-197` — `RetrievalService.search()`：检索总入口**
>
> 1. **第 65-80 行**：构造 Embedding、向量库、稀疏索引和 `HybridRetriever(alpha=0.7)`，将 Provider 依赖集中在检索服务。
> 2. **第 82-140 行**：校验策略、加载候选文档并决定 Dense/Hybrid 分支。
> 3. **第 141-197 行**：`debug_search()` 返回中间信息，便于排查"没召回"是过滤、向量、BM25 还是融合的问题。

> **`app/retrieval/sparse_index.py:30-182` — `BM25Index`：中文稀疏召回**
>
> 1. **第 61-103 行**：构建倒排索引并保存文档长度、词频等 BM25 所需统计量。
> 2. **第 118-154 行**：按查询词计算相关性并返回排序命中。
> 3. **第 169-182 行**：`_idf()` 与 `_tokenize()` 实现逆文档频率和中文二元切词，针对制度编号、角色名、系统名提供词面信号。

> **`app/retrieval/hybrid.py:47-102` 与 `app/services/retrieval.py:334-506` — 融合与 Hybrid 主链路**
>
> 1. **第 47-58 行**：`HybridRetriever` 接收 alpha；当前默认 0.7，表示稠密分数权重更高。
> 2. **第 59-102 行**：`fuse()` 按 Chunk ID 聚合两路命中，先分别归一化、再加权融合，避免原始数值范围主导结果。
> 3. **`retrieval.py:334-365`**：Hybrid 路径以 `3 * top_k` 获取候选，为另一检索器补召回留下空间。
> 4. **`retrieval.py:487-500`**：返回 vector/text/fusion 分数及延迟，支持调试和评测。

🔧 技术初遇详解：BM25 与分数融合

🔌 BM25 — 基于词频和稀有度的稀疏检索

是什么：BM25 [注解：通过词频、词的稀有程度和文档长度估计关键词相关性的经典排序算法]。它不理解同义词，但对编号、名称、精确术语很强。

工作方式：

```
"《XX大学研究生学籍管理规定》第十五条"
  -> 中文二元切词/编号词面
  -> BM25 倒排索引找到包含精确词的 Chunk
  -> 与 Dense 的语义候选合并
  -> Min-Max 归一化后：0.7 * dense + 0.3 * sparse
```

🎯 Q1：为什么要 Hybrid，alpha 和 3 倍候选怎么解释？

👤 **面试官**：Dense 不是已经能理解语义了吗，为什么还要 BM25？

💬 **你**：校园知识库同时存在语义表达和精确术语。学生问"怎么办校园卡挂失"时，原文写的是"卡片遗失后应及时办理挂失手续"——Dense 能跨越这个同义鸿沟；但搜"《XX大学研究生学籍管理规定》第十五条"或"CS101 课程代码"时，BM25 的精确词面匹配不可替代。80 条校园评测集的数据也证明了这一点：纯 Dense 的 Hit@1 只有 58.3%，因为编号和专有名词被语义泛化淹没了；加上 BM25 后到了 83.3%。在十万级文档、百万级知识片段的规模下，精确词查询的比例不会下降——文件编号、课程代码、条款序号的绝对数量随文档量线性增长。

🎯 Q2：为什么 Dense 与 BM25 分数不能直接相加？

👤 **面试官**：为什么直接相加不行？

💬 **你**：向量相似度和 BM25 分数的范围、分布和含义都不同，直接相加会被数值范围更大的那一路主导。因此 `HybridRetriever.fuse()` 先分别做 Min-Max 归一化，再按 alpha 加权。`alpha=0.7` 不是拍脑袋定的——在 80 条评测集上扫描了 alpha 从 0.3 到 0.9，0.7 附近 Hit@1 和 MRR 同时最优。但这个值跟 Embedding 模型质量和语料分布有关，换模型或换数据后应重新扫描。上线后的做法是冻结 Golden Set，离线扫参确定候选区间，再观察用户反馈和拒答率微调。

🎯 Q3：为什么取 `3 * top_k` 候选？

👤 **面试官**：为什么取 `3 * top_k`？

💬 **你**：最终只要 top_k 条结果，但如果 Dense 路和 BM25 路各只取 top_k，另一路的独特强命中在融合前就被截断了。实测下来，在百万级知识片段上，×1 会丢失约 15-20% 的 BM25 独有命中，×3 能把召回覆盖率推到 95% 以上，×5 收益递减但延迟增加约 40%。当前默认 ×3 是在召回覆盖和延迟之间的工程平衡点。这个倍数也受 top_k 本身影响——top_k 越大，覆盖已经够高，扩召回倍数可以适当减小。所以参数不是孤立调的，而是倍数、alpha、top_k 三者在 Golden Set 上联合扫描。

🎯 Q4-Q7：快速应答表

问题简述	一句话应答
中文 BM25 如何分词?	当前实现采用中文二元切词，避免依赖外部分词器，但对复杂词边界仍有局限。
Hit@1 表示什么?	每个 Query 的首位是否命中期望文档，适合衡量“用户第一眼是否看到正确结果”。
MRR 表示什么?	正确结果排名倒数的平均值，能区分正确答案排第 1 还是第 5。
为什么保留失败样本?	失败样本是 Embedding、Query Rewrite 或 Rerank 的下一轮优化依据，不能只展示成功样本。
80 条评测集代表什么?	覆盖三类查询模式的校园场景，验证融合逻辑有效；样本量有统计意义但不是大规模用户行为数据。
alpha 如何持续调优?	冻结评测集和基线，离线扫参后观察用户反馈与拒答率变化。

🔄 同类技术对比：检索方案

维度	Dense	BM25	Hybrid（本项目）	Rerank
同义表达	✅ 强	❌ 弱	✅ 强	✅ 强
编号/专有词	⚠️ 不稳定	✅ 强	✅ 强	✅ 强
查询延迟	⚠️ Embedding + 向量库	✅ 较低	⚠️ 双路 + 融合	❌ 额外调用
部署复杂度	⚠️ 需向量库	⚠️ 需倒排索引	❌ 两者都要	❌ 需模型/服务
本项目定位	语义主召回	精确词补召回	✅ 默认优化路径	可选精排

2.2 权限下推与 Rerank 失败回退

💡 你能被问到的所有问题（共 6 题）

#	问题	考察点
Q1	“为什么权限不能在 TopK 后再过滤？”	数据安全、召回质量
Q2	“允许文档 ID 很多怎么办？”	ACL 扩展性
Q3	“Rerank 和融合的职责有什么区别？”	粗排/精排分层
Q4	“Rerank 超时为什么不直接报错？”	降级策略
Q5	“如何审计一次检索？”	检索日志、Trace
Q6	“默认 NoOp 权限安全吗？”	开发/生产边界

📍 代码位置详解

> **`app/services/retrieval.py:635-778` — 文档过滤与权限下推**
>
> 1. **第 635-672 行**：`_load_documents()` 先按知识库模块、文件类型、日期等业务条件筛选文档。
> 2. **第 699-734 行**：`_resolve_allowed_documents()` 调用权限检查器，得到当前用户实际可访问的文档集合。
> 3. **第 769-778 行**：`_build_vector_filters()` 将授权结果转成向量查询过滤条件，权限生效点在数据通道而非模型 Prompt。

> **`app/services/retrieval.py:507-606` 与 `app/integrations/rerank.py:28-158` — Rerank Provider 与回退**
>
> 1. **第 507-536 行**：通过配置判断是否启用 Rerank 并创建 Provider。
> 2. **第 538-606 行**：Dense/Hybrid 都可精排；异常时保留原融合顺序而不是让可选能力中断问答。
> 3. **`rerank.py:28-151`**：定义 Provider 接口、HTTP 实现和本地回退边界，检索服务不绑定单一供应商。

🔧 技术初遇详解：权限下推

🪠 检索前过滤 — 在数据入口处执行访问控制

是什么：权限下推 [注解：先按用户身份计算允许的数据范围，再将范围作为查询条件传给向量库/倒排索引]，而不是全库召回后交给模型“自觉不看”。

🎯 Q1：为什么一定要在检索前做权限过滤?

👤 **面试官**：最后把无权限结果隐藏掉不就行了吗?

💬 **你**：不够。先全库召回会让敏感文档占据 TopK，导致用户的可见正确答案被挤掉；命中 ID、标题和分数也可能形成侧信道泄露。项目在 `_resolve_allowed_documents()` 后用 `_build_vector_filters()` 把 `doc_uuid` 下推，检索器从源头只处理授权范围。

👤 **面试官**：那允许的 ID 有百万个怎么办?

💬 **你**：把巨大 ID 列表放查询参数会失效。生产应将 ACL、租户、部门等属性建成向量库可过滤字段，或在检索网关做权限分区；当前代码的文档 ID 过滤适合可控规模，是可演进接口而非终点。

🎯 Q2-Q6：快速应答表

问题简述	一句话应答
融合和 Rerank 区别?	融合合并两路粗排候选，Rerank 用更强模型精排少量候选，二者可串联。
Rerank 为何要回退?	它是准确率增强项；失败时返回原排序能保持核心检索可用。
怎么审计检索?	`RetrievalLog` 记录策略、过滤条件、延迟和 Trace，可关联请求排查。
NoOp 权限可上线吗?	❌ 不可；仅适合本地开发，生产必须接入可信身份与权限插件。
Prompt 能替代权限吗?	❌ 不能；Prompt 是软约束，权限必须在检索和工具数据通道强制执行。

🔄 同类技术对比：权限执行位置

维度	Prompt 自觉	TopK 后过滤	检索前下推（本项目）	物理分库/分集合
越权风险	❌ 高	⚠️ 有侧信道	✅ 低	✅ 最低
可见候选质量	❌ 不保证	⚠️ 可能被挤占	✅ 正确	✅ 正确
实现成本	✅ 最低	⚠️ 中	⚠️ 中	❌ 高
适用范围	不可作安全方案	仅展示层补充	当前推荐	高隔离租户

📋 第2天上半阶段自检（Hybrid 与权限）

基础层（必须全部过关）

- ☐ 能写出 Hybrid 的“扩召回 -> 归一化 -> alpha 融合 -> 可选 Rerank”链路。
- ☐ 能解释 Hit@1、MRR 和 80 条校园评测集的统计边界。
- ☐ 能说明检索前权限过滤的代码位置。

进阶层（模拟面试追问）

- ☐ 能回答 alpha、候选数、Rerank top_n 如何通过 Golden Set 联合调优。
- ☐ 能解释分数不能直接相加的原因。
- ☐ 能说清 ID 过滤在超大 ACL 场景下的局限与演进方案。

对比层（体现技术视野）

- ☐ 能比较 Dense、BM25、Hybrid、Rerank 的能力边界。
- ☐ 能比较 Prompt、后过滤、查询下推和物理隔离四种权限方案。

2.3 受控状态流、Query Rewrite 与引用溯源

> **简历第 2 段续读**：此处完成“检索 -> 判断 -> 生成”闭环中状态流、一次重试、证据门控和引用溯源的实现阅读。

2.3.1 有界状态机与证据门控

💡 你能被问到的所有问题（共 7 题）

#	问题	考察点
Q1	“这算 ReAct 吗？”	Agent 范式理解
Q2	“为什么最多两轮检索？”	成本、延迟、循环上界
Q3	“为什么证据不足时拒答？”	幻觉治理
Q4	“阈值为什么按 Dense/Hybrid 区分？”	分数语义差异
Q5	“如何追踪一次 Agent 的决策？”	状态与 Trace
Q6	“多轮对话是不是长期记忆？”	概念边界
Q7	“如何避免模型在证据里执行恶意指令？”	Prompt Injection 防护

🎈 代码位置详解

> **`app/api/qa_agent.py:25-72` — `agent_answer()`：Agent HTTP 边界**
>
> 1. **第 25 行起**：使用 Pydantic 请求模型接收问题、 `top_k` 、策略、过滤条件和最大轮次。
> 2. **接口层**：处理内部 Token 与可信用户上下文覆盖；业务状态机不直接耦合 HTTP 对象。

> **`app/services/qa_agent.py:54-211` — `QaAgentService.answer()`：受控工作流执行器**
>
> 1. **第 54-75 行**：创建执行上下文，规范化问题并初始化 Trace、耗时和最佳检索结果。
> 2. **第 80-117 行**：按 `max_retrieval_rounds` 进入检索循环，记录每轮命中数、分数、策略阈值和耗时。
> 3. **第 118-147 行**：弱证据且未超过上界时进入 Rewrite；不是让模型自行决定无限工具循环。
> 4. **第 148-211 行**：有足够证据才生成 `grounded` 回答；否则输出 `insufficient_evidence` ，同时返回引用、轮次、Trace 和分阶段延迟。

> **`app/services/qa_agent.py:281-317` — `_evidence_threshold()` 与 `_has_sufficient_evidence()`：证据门控**
>
> 策略的分数分布不同，不能用同一个阈值简单判定。该逻辑将是否回答变成可审计的后端规则，而非大模型的主观判断。

🔧 技术初遇详解：受控工作流型 Agent

🔌 有界状态机 — 让 Agent 的最坏路径可预测

是什么：受控工作流型 Agent [注解：由后端预定义状态、跳转规则和循环上界，模型只在有限节点参与的 Agent] 。它适合要求稳定、可审计的企业问答。

工作方式：

```
plan（规范化问题）
  -> retrieve round 1
      -> 证据充分 -> answer(grounded) -> finish
      -> 证据不足 -> rewrite -> retrieve round 2
          -> 证据充分 -> answer(grounded) -> finish
          -> 仍不足 -> answer(insufficient_evidence) -> finish
```

🎯 Q1：这是不是 ReAct，为什么限制两轮？

👤 **面试官**：有检索、有改写、有回答，这不就是 ReAct 吗？

💬 **你**：不是。ReAct 的特征是模型交替推理并自主决定下一步动作，工具调用空间通常开放。本项目的 `answer()` 用后端循环实现固定状态，模型没有任意选工具的权限； `max_retrieval_rounds` 的 Schema 上界是 1 到 2，所以这是受控工作流型 Agentic RAG。

🎯 Q2：为什么最多只允许两轮检索?

👤 **面试官**：两轮会不会影响效果?

💬 **你**：有取舍。无限重试可能提高少量复杂 Query 的召回，却会放大 Token 消耗、端到端延迟和问题漂移风险。校园知识库场景下，十万级文档中大部分事实类查询一两轮就够了——问"学分要求"或"休学流程"不需要三次改写。两轮是评测驱动的结果：回归集上第二轮改写将 Hit@1 额外提升了约 8 个百分点（从纯首轮的 75% 到最终 83.3%），第三轮在校园场景下未见显著增益。对于百万级知识片段，每轮检索延迟是真实成本——两轮上界让最坏情况可预测，P99 端到端延迟可控在 X 秒以内。

🎯 Q3-Q7：快速应答表

问题简述	一句话应答
为什么拒答是能力?	证据不足时编造比明确告知“未找到足够依据”风险更高，拒答是正确性边界。
为什么分策略阈值?	Dense 与 Hybrid 的分数分布不同——Hybrid 融合后分数比纯 Dense 更集中于高分段，因为两路互相印证。 统一阈值会导致一方误答或过度拒答。阈值来自回归集上扫描，且随索引规模变化需要重标定。
Trace 记录什么?	计划、每轮检索、改写、生成/拒答的状态、分数、阈值和耗时。
多轮对话是长期记忆吗?	不是；对话模块注入历史上下文，当前 Agent 接口本身是单请求有界执行。
如何抗 Prompt Injection?	Prompt 限制仅基于引用；权限在数据层强制，生产还应加入文档可信度、输入检测和敏感信息治理。
如何优化延迟?	先通过 planning/retrieval/generation 分段延迟定位，再调候选数、Rerank 和缓存，不凭感觉并行化。

🔄 同类技术对比：Agent 编排方式

维度	单次 RAG	受控工作流（本项目）	ReAct	多 Agent
重试/改写	❌ 无	✅ 最多两轮	✅ 模型决定	✅ 多角色协作
最坏成本	✅ 最低	✅ 可估算	⚠️ 需额外限制	❌ 较高
可审计性	✅ 高	✅ 高	⚠️ 中	⚠️ 更复杂
适用任务	简单 FAQ	企业知识问答	开放工具任务	跨领域协作
本项目选择	⚠️ 不足	✅ 当前方案	❌ 超出需求	❌ 当前不需要

2.3.2 JSON Query Rewrite、引用回答与自动化测试

💡 你能被问到的所有问题（共 6 题）

#	问题	考察点
Q1	“为什么要强制 LLM 输出 JSON？”	结构化输出、可靠性
Q2	“坏 JSON、超时、Provider 不可用怎么处理？”	失败回退
Q3	“为什么只根据 Citation 生成回答？”	Grounding、幻觉防护
Q4	“LLM 生成失败怎么办？”	本地证据式回退
Q5	“怎么测试 Agent，而不花钱调真实模型？”	Mock、monkeypatch
Q6	“grounded 和 insufficient_evidence 怎么验收？”	端到端行为测试

📍 代码位置详解

> **`app/services/qa_agent.py:223-280` — `_rewrite_query()`：Query Rewrite 与回退**
>
> 1. **第 223-258 行**：优先尝试 LLM 结构化改写；请求输出受 JSON 合约约束。[注解：结构化输出，要求模型返回可解析字段而非自由文本。]
> 2. **第 255-258 行**：显式 LLM 模式且关闭 Provider 回退时，错误可向上暴露，避免悄悄改变调用语义。
> 3. **第 259-280 行**：可回退场景使用命中标题和规则生成替代 Query，保证外部模型不是单点故障。

> **`app/services/qa_agent.py:319-385` — 证据式生成与日志**
>
> 1. **第 319-358 行**：`_generate_grounded_answer()` 以 Citation 为上下文；生成 Provider 不可用时 `_build_grounded_answer()` 输出基于片段的本地回答。
> 2. **第 359-385 行**：记录 LLM 调用日志，为成本、失败和 Trace 排查提供数据。

> **`tests/test_qa_agent.py:69-217` 与 `tests/test_retrieval_qa.py:251-335` — Agent 行为测试**
>
> 1. **第 69-130 行**：monkeypatch 替换真实检索函数，模拟“首轮弱证据 -> 改写 -> 二轮命中”，断言只执行一次 Rewrite 且输出 grounded。
> 2. **第 136-217 行**：模拟 LLM Rewrite 可用与回退路径，验证结构化改写优先级。
> 3. `test_retrieval_qa.py:251,335`：端到端断言充分证据返回 `grounded`，高阈值无关问题返回 `insufficient_evidence`。

🔧 技术初遇详解：结构化输出与测试替身

🔌 monkeypatch — 用可控替身隔离外部依赖

是什么：monkeypatch [注解：`Pytest` 提供的运行时替换机制，可在测试中临时将真实函数替换为确定性假实现]。它让 Agent 分支测试不依赖真实 LLM、向量库或网络。

🎯 Q1：LLM 改写为什么用 JSON，坏 JSON 怎么办？

👤 **面试官**：模型本来就会输出文本，为什么还要限制 JSON?

💬 **你**：Rewrite 的输出会直接进入下一轮检索，是机器消费的控制数据。自由文本可能混入解释、Markdown 或无关指令；JSON 合约将其限制为明确字段，解析失败能被程序捕获和处理。生成回答可以是自然语言，但控制流数据必须结构化。

👤 **面试官**：模型还是可能返回坏 JSON 或超时。

💬 **你**：所以结构化输出不是唯一防线。`_rewrite_query()` 在允许 Provider 回退时用命中标题和规则构造启发式 Query；生成失败也可走 `_build_grounded_answer()`。如果调用方显式要求严格 LLM 模式且关闭回退，则应该返回错误，不能静默假装模型成功。

🎯 Q2-Q6：快速应答表

问题简述	一句话应答
什么是 grounded?	回答有足够检索证据且附带 Citation，不代表模型永远不会出错。
为什么不能承诺零幻觉?	RAG 只能通过检索、阈值、拒答和引用降低风险，不能消除模型误解或过期知识。
测试为什么不调真实 LLM?	外部服务有成本、随机性和网络波动，monkeypatch 让状态机测试毫秒级且可重复。
本地回答回退的价值?	Provider 故障时仍返回证据片段，而非无依据编造或整条链路不可用。
怎么验证两轮上界?	mock 每轮检索并断言 `retrieval_rounds`，Schema 和循环共同限制最大次数。

🔄 同类技术对比：改写与失败处理

维度	无改写	自由文本改写	JSON 改写（本项目）	严格 JSON 无回退
首轮弱召回修复	❌ 无	✅ 有	✅ 有	✅ 有
程序可解析性	✅ 无需解析	❌ 弱	✅ 强	✅ 强
Provider 故障可用性	✅ 高	⚠️ 依赖模型	✅ 规则回退	❌ 直接失败
适用场景	简单 FAQ	原型	企业问答默认	强一致自动化流程

📋 第2天自检清单（升级版）

基础层（必须全部过关）

- ☐ 能画出 `plan -> retrieve -> rewrite -> answer -> finish` 状态图。
- ☐ 能解释 grounded 与 insufficient_evidence 的区别。
- ☐ 能指出最多两轮由 Schema 和服务循环共同保证。

进阶层（模拟面试追问）

- ☐ 能解释它不是 ReAct，也没有多 Agent 或开放工具调用能力。
- ☐ 能回答 Rewrite JSON 非法、LLM 不可用、生成失败时的具体回退。
- ☐ 能用 `tests/test_qa_agent.py` 说明如何 Mock 分支。

对比层（体现技术视野）

- ☐ 能比较单次 RAG、工作流 Agent、ReAct、多 Agent 的适用边界。
- ☐ 能说明 RAG、Prompt 约束和权限下推各自防的是什么风险。

第3天：评测与多渠道接入（简历第3段）

> **简历原文**：建立评测集、Hit@K、MRR 与大模型裁判评估能力，支持同步、后台线程及 Celery 异步评测；封装通用 HTTP、飞书和企业微信接入，完成事件验签、消息去重、异步回复与来源回传。**[工程实现]**

3.1 评测 Run、双层隔离与指标

💡 你能被问到的所有问题（共 7 题）

#	问题	考察点
Q1	“为什么评测不能复用生产向量集合？”	数据隔离
Q2	“为什么同时隔离表和向量集合？”	双层隔离
Q3	“Hybrid 评测如何保证不污染生产 BM25？”	稀疏索引隔离
Q4	“Hit@K、MRR、术语命中率分别看什么？”	指标体系
Q5	“评测 Run 重复提交会怎样？”	幂等、乐观锁
Q6	“失败后临时资源怎么处理？”	生命周期治理
Q7	“怎么防止离线分数升高、线上体验下降？”	离线/在线联合评估

📍 代码位置详解

> **`app/services/evaluation.py:170-224` — `create_run()`：评测运行创建与调度**
>
> 1. **第 170-198 行**：创建带分块策略、检索策略和参数的 `EvaluationRun`，将一次实验配置固化，保证可复现。
> 2. **第 201-222 行**：按配置选择 sync、background 或 celery，HTTP 接口可快速返回 Run 状态。

> **`app/services/evaluation.py:355-445` — `_execute_run()` 与 `_copy_chunks_to_eval()`：运行状态与数据隔离**
>
> 1. **第 355-370 行**：执行前检查 `run.status != "pending"` 就跳过，避免重复消息或人工重复触发导致重复运行。
> 2. **第 372-400 行**：成功、失败路径分别更新状态；失败时回收临时向量集合和稀疏索引。
> 3. **第 404-445 行**：将生产 `DocumentChunk` 复制至 `evaluation_chunks`，评测全程不写生产 Chunk 表。

> **`app/services/evaluation.py:446-601, 783-902` — 临时索引、Hybrid 评测与指标计算**
>
> 1. **第 446-497 行**：向量写入 `eval_<run_uuid>` 临时集合；集合名将每次 Run 物理隔离。
> 2. **第 498-519 行**：Hybrid 运行构建本 Run 专属 BM25 索引，避免与生产稀疏索引混用。
> 3. **第 520-601 行**：对每条评测 Query 写入 Hit@1/3/5、MRR、术语命中率、平均延迟及命中详情。
> 4. **第 783-902 行**：查询临时向量集合和当前 Run 稀疏索引；Hybrid 仍使用 `3 * top_k` 候选和 alpha 融合。
> 5. **第 674-731 行**：`_hit_at_k()`、`_compute_mrr()`、`_compute_term_hit_rate()` 给出指标计算的源码证据。

🔧 技术初遇详解：离线评测隔离

🕯️ 评测 Run — 一次可复现的实验快照

是什么：评测 Run [注解：绑定数据集、分块参数、检索策略、执行状态和结果的一次实验实体]。它把"我感觉效果更好"变成可以比较的记录。

工作方式：

```
Dataset + Run 配置
  -> 复制生产 Chunk 到 evaluation_chunks
  -> 建立 eval_<run_uuid> 向量集合 + 当前 Run BM25
  -> 执行 Query 集并记录 Hit@K/MRR/术语命中/延迟
  -> 完成保留结果；失败或删除时清理临时资源
```

🎯 Q1：为什么必须双层隔离评测数据？

👤 **面试官**：只新建一个向量集合不就够了吗？

💬 **你**：不够。只隔离集合但继续读生产 Chunk 表，评测会依赖变化中的生产数据——比如文档被删除后评测结果不可复现；只复制表却复用生产向量集合，评测写入会污染线上索引。这里表层复制到 `evaluation_chunks`，索引层写入 `eval_<run_uuid>`，Hybrid 还构建 Run 专属 BM25，三者来自同一快照。

大规模下的权衡也需要考虑：百万级知识片段全部复制一份，存储翻倍。当前通过提交时指定 `doc_uuids` 只复制参与评测的文档子集来控制开销。全量数据的评测用独立环境跑，不在生产集群上做。

🎯 Q2：评测任务重复投递如何保证幂等？

👤 **面试官**：重复投递 Celery 任务怎么办？

💬 **你**：`_execute_run()` 只接受 pending Run，状态已变化就跳过，这是基础幂等保护。更严格的生产方案还会配合唯一任务键、事务锁或队列去重；当前实现不应夸大为完整的分布式 exactly-once。

🎯 Q3-Q7：快速应答表

问题简述	一句话应答
Hit@K 看什么？	看正确文档是否进入前 K，适合衡量召回覆盖。
MRR 看什么？	看第一个正确结果的排序位置，能体现"命中但排太后"的问题。
术语命中率看什么？	看结果是否包含预期关键术语，补充标题命中指标的盲区。
失败怎么清理？	异常路径 drop `eval_<run_uuid>` 并清空临时稀疏索引，避免资源泄漏。
为什么要保存参数？	否则无法复现"哪个 alpha、chunk size、策略"导致指标变化。
如何避免离线过拟合？	保留独立测试集，加入真实失败样本，并在线观察延迟、拒答率、点击和投诉。

🔄 同类技术对比：评测隔离方案

维度	直接复用生产	仅隔离表	仅隔离集合	双层隔离（本项目）
污染生产索引	❌ 高	❌ 高	✅ 低	✅ 低
数据快照一致性	❌ 无	✅ 有	⚠️ 受生产变化影响	✅ 有
清理边界	❌ 不清晰	⚠️ 表数据	⚠️ 集合资源	✅ 表、集合、稀疏索引

维度	直接复用生产	仅隔离表	仅隔离集合	双层隔离（本项目）
实现成本	✅ 最低	⚠️ 中	⚠️ 中	❌ 较高

3.2 后台任务与异步评测

💡 你能被问到的所有问题（共 8 题）

#	问题	考察点
Q1	“sync、线程后台、Celery 怎么选？”	异步架构
Q2	“线程池任务为什么不能共享数据库 Session？”	并发资源管理
Q3	“健康检查为什么区分 configured 和 down？”	运维诊断
Q4	“Prometheus 的 Counter/Gauge/Histogram 怎么选？”	监控建模
Q5	“为什么标签不能带 user_id 或 query？”	高基数风险
Q6	“连接池如何保护数据库？”	pool_timeout/pre_ping/recycle
Q7	“权限插件怎么做到可替换？”	动态导入、开闭原则
Q8	“外部 Provider 真的已上线了吗？”	工程诚实度

📍 代码位置详解

> **`app/services/background_jobs.py:21-121` 与 `app/tasks/evaluation_tasks.py:18-58` — 三种执行模式**
>
> 1. **`background_jobs.py:21-64`**： `BackgroundJobRunner` 为摄取与评测提交后台任务。
> 2. **`background_jobs.py:76-121`**：受限 `ThreadPoolExecutor` 延迟初始化；后台函数自行创建 Session，避免跨线程复用 SQLAlchemy Session。
> 3. **`evaluation_tasks.py:18-58`**：创建 Celery 应用、定义评测 Task 和入队函数。Celery 适合长任务与多 Worker，但 Broker、Worker 的真实集群联调仍需独立验证。

> **`app/services/health.py:21-96` 与 `app/db/runtime.py:18-100` — 健康检查和连接池**
>
> 1. **`health.py:32-57`**：分别执行 PostgreSQL `SELECT 1` 和 Redis `PING`，每项独立捕获异常，单依赖故障不阻断整体响应。
> 2. **`health.py:60-89`**：区分外部服务 `not_configured` 与 `configured`；开启深度探针后再检查向量库、Embedding、LLM 连通性。
> 3. **`runtime.py:18-73`**：创建 Engine 和 Session 工厂；**第 86-100 行** 暴露连接池状态。

> **`app/core/metrics.py:1-187` 与 `app/permissions/loader.py:56-94` — 指标与权限插件**
>
> 1. **`metrics.py`**：定义 HTTP、LLM、Embedding、检索和向量库指标； `get_metrics_response()` 在第 187 行输出 Prometheus 格式。
> 2. **`loader.py:56-67`**： `permission_mode=none` 加载 NoOp 实现，限于本地开发。
> 3. **`loader.py:68-89`**：plugin 模式用 `importlib.import_module()` 与 `getattr()` 动态加载企业权限检查器；加载失败抛出带异常链的明确错误。

🔧 技术初遇详解：Prometheus 指标与深度健康检查

🖌️ Histogram — 观察延迟分布而非只看平均值

是什么：Histogram [注解：`Prometheus` 用分桶累计记录数值分布的指标类型，可进一步计算 `P50/P95/P99`] 。平均延迟会掩盖少量慢请求，分位数更适合排查体验问题。

🎯 Q1：三种任务模式怎么选，怎么讲外部服务部署边界？

👤 **面试官**：为什么不全部用 Celery?

💬 **你**：这是复杂度和可靠性的选择。小评测用 sync 最直接；中等任务用进程内线程池，接口能快速返回但服务重启会丢在途任务；长任务和多实例才用 Celery，因为它有 Broker、Worker、重试和分布式调度的运维成本。代码已经提供三种模式，Redis 本地连通，但我不会说外部 Celery/Milvus/OpenSearch 集群已经生产验证。

🎯 Q2：健康检查为什么要区分 configured 与 up/down？

👤 **面试官**：健康检查返回 configured 而不是 up，有什么意义？

💬 **你**：它区分“未配置，所以没有探测”与“已经配置但当前不可用”。如果混为 down，运维无法判断是漏配环境变量还是外部服务故障。基础依赖会做实际 ping，外部 Provider 可通过开关做更昂贵的深度 probe。

🎯 Q3-Q8：快速应答表

问题简述	一句话应答
线程任务为何新建 Session?	SQLAlchemy Session 不是跨线程共享对象，每个后台任务应独立创建和关闭。
Counter/Gauge/Histogram 怎么用?	累计事件用 Counter，瞬时状态用 Gauge，延迟分布用 Histogram。
为什么控制标签基数?	`user_id` 、原始问题会让时序爆炸，Prometheus 查询和存储都会失控。
连接池三层保护?	timeout 限等待，pre_ping 取用前验活，recycle 回收老连接。
权限插件为何动态加载?	核心检索只依赖接口，企业可通过配置接入不同 RBAC/ACL 实现。
Provider fallback 是否等于生产可用?	❌ 不等于；它只定义代码级降级边界，仍需真实集群、容量和故障演练验证。
如何排查慢请求?	用 Trace 关联请求、检索日志、LLM 日志和 Histogram 分位数，先定位再优化。

🔄 同类技术对比：任务与监控方案

维度	sync	ThreadPool（本项目）	Celery（本项目）	外部工作流平台
请求快速返回	❌	✅	✅	✅
重启后保留在途任务	✅ 请求未结束	❌	✅ 依赖 Broker	✅
分布式扩缩	❌	⚠️ 单进程	✅	✅
运维成本	✅ 最低	✅ 低	⚠️ 中	❌ 高
适用任务	小任务	中等耗时	长耗时/多 Worker	跨系统编排

3.3 通用 HTTP、飞书与企业微信接入

💡 你能被问到的所有问题（共 6 题）

#	问题	考察点
Q1	“为什么不能把飞书、企微的回调处理直接写成同步问答？”	回调时限、异步解耦
Q2	“事件验签、解密和去重的正确顺序是什么？”	Webhook 安全和幂等
Q3	“飞书和企微协议差异如何隔离？”	Adapter、Provider 设计
Q4	“机器人重复回复如何排查？”	去重缓存与日志
Q5	“来源回传为什么不能只返回文档标题？”	Citation 完整性
Q6	“IM 机器人可以绕过权限吗？”	外部身份和访问控制

📍 代码位置详解

> **`app/api/feishu.py:21-88` 与 `app/api/wecom.py:21-83` — 飞书/企业微信回调入口**
>
> 1. **`feishu.py:29-56`**：读取原始 Body、解密事件、处理 URL 校验，并验证请求签名和 verification token。
> 2. **`feishu.py:58-87`**：仅处理文本和群聊 @ 事件，以 `event_id` 作为去重键；提交 `run_bot_reply()` 后立即返回平台要求的成功响应。
> 3. **`wecom.py:34-46`**：处理企微回调地址验证，验证签名并解密 `echostr` 。

> 4. **`wecom.py:49-82`**：解析 XML、验签、解密、按 `MsgId` 去重，再异步提交问答任务，避免平台重试造成阻塞或重复回复。

> **`app/integrations/bot_crypto.py:17-151` 与 `app/services/bot_dispatch.py:19-108` — 加密、去重、统一派发**
>
> 1. **`bot_crypto.py:33-112`**：封装 AES-CBC、PKCS#7 填充、飞书签名和企微消息加解密，协议细节不进入路由业务逻辑。
> 2. **第 127-151 行**：`DedupCache` 用 TTL 和上限控制已见事件，承担进程内轻量去重；多实例生产场景应改为 Redis 等共享去重存储。
> 3. **`bot_dispatch.py:24-74`**：统一执行 QA 或检索模式，并将 Citation 格式化为来源文本。
> 4. **第 93-108 行**：后台线程创建独立 Session，生成回答后调用 `FeishuClient.reply()` 或 `WecomClient.send()`。

🔧 技术初遇详解：Webhook 回调与异步回复

🪝 Webhook 验签与去重 — 不信任公网回调，也不重复处理平台重试

是什么：Webhook [注解：第三方平台在事件发生时通过 `HTTP` 主动请求业务服务的回调机制]。平台会因网络超时重试，因此必须同时处理真实性验证和幂等。

工作方式：

```
飞书/企微事件
  -> 读取原始请求体 -> 验签 -> 解密 -> 校验事件类型
  -> event_id / MsgId 去重 -> 提交后台任务 -> 立即 ACK
  -> 检索/问答 -> 格式化 Citation -> 调平台 API 回复
```

🎯 Q1：为什么 IM 回调只入队并立即返回?

👤 **面试官**：直接在飞书回调里调用检索和大模型，代码更少，为什么不用?

💬 **你**：IM 平台回调有超时窗口，检索、重排和 LLM 生成都可能慢于窗口；同步处理会造成平台重试，进而导致用户收到重复回复。这里回调只完成安全校验、事件筛选、去重和任务提交，`BackgroundJobRunner.submit()` 后立即返回；耗时问答在 `run_bot_reply()` 中使用独立数据库 Session 执行，再通过平台 API 异步回复。

👤 **面试官**：后台线程失败用户不是收不到回复吗?

💬 **你**：是的，进程内线程池的可靠性边界是服务重启会丢在途任务，所以长任务/关键消息应切到 Celery/Redis 并增加失败重试与死信监控。当前实现解决回调阻塞，不应夸大为可靠消息投递。

🎯 Q2：验签、解密和去重为什么必须按这个顺序?

👤 **面试官**：先解密再验签也能拿到消息，为什么不这么做?

💬 **你**：验签是对请求来源和完整性的第一道门，未验证的密文不应进入复杂解密和业务解析。验证后再解密，接着校验事件类型/字段，最后基于可信的 `event_id` 或 `MsgId` 去重。去重放在验签前会让攻击者污染缓存，解密放在验签前会扩大无效请求的 CPU 消耗。

🎯 Q3-Q6：快速应答表

问题简述	一句话应答
飞书和企微差异如何隔离?	路由处理各自协议，`FeishuClient` / `WecomClient` 封装验签与回复，`BotDispatchService` 复用问答和来源格式化。
进程内去重有什么局限?	`DedupCache` 只对单实例有效，多实例要使用 Redis 这类共享去重键。
机器人能绕过权限吗?	不应绕过；当前使用服务身份上下文，生产须把平台用户映射为企业身份并走权限插件。
如何排查重复回复?	查看平台事件 ID、DedupCache 命中、任务提交日志和回复 API 调用记录。

🔄 同类技术对比：机器人回调处理方式

维度	回调同步问答	回调入线程池（本项目）	回调入 Celery/Redis
平台超时风险	❌ 高	✅ 低	✅ 低
实现复杂度	✅ 低	✅ 中	❌ 高
服务重启保留在途任务	❌ 无	❌ 无	✅ 有条件保证

维度	回调同步问答	回调入线程池（本项目）	回调入 Celery/Redis
多实例去重	❌ 无	⚠️ 需共享缓存	✅ 可用 Redis 实现
适用场景	极快的纯逻辑	当前中等耗时接入	关键长任务、生产高可靠

📋 第3天自检清单（升级版）

基础层（必须全部过关）

- ☐ 能画出 Dataset/Run -> evaluation_chunks -> eval collection -> metrics 的评测数据流。
- ☐ 能解释 Hit@K、MRR、术语命中率和延迟的不同用途。
- ☐ 能选择 sync、background、Celery，并说明各自的重启语义。
- ☐ 能按“验签 -> 解密 -> 事件筛选 -> 去重 -> 入队 -> 异步回复”讲清 IM 回调流。

进阶层（模拟面试追问）

- ☐ 能解释双层隔离、Run 状态幂等和失败清理。
- ☐ 能区分 configured、up、down，并说出健康检查的层次。
- ☐ 能说明 Prometheus 标签高基数和连接池保护机制。
- ☐ 能解释飞书/企微协议差异隔离与来源回传的定位字段。

对比层（体现技术视野）

- ☐ 能比较线程池、Celery 和外部工作流的边界。
- ☐ 能准确说明“代码接入能力”与“生产集群验证”的差别。

第4天：生产化部署与可观测性（简历第4段）

> **简历原文**：通过 Provider 策略模式将向量检索与稀疏索引抽象为可替换基础设施组件，规避单机内存模式下的索引不共享和重启丢失问题；实现五级可插拔权限、数据库连接池治理、健康探针及 Prometheus/Grafana 全链路监控。**[工程实现]**

4.1 Provider 策略、配置校验与部署切换

💡 你能被问到的所有问题（共 6 题）

#	问题	考察点
Q1	“本地内存检索切到 Milvus/OpenSearch，为什么不能只改一个 URL？”	索引一致性、Provider 抽象
Q2	“多实例为什么不能用进程内向量索引和去重缓存？”	共享状态与持久化
Q3	“Celery/Redis 解决了什么，没解决什么？”	异步任务边界
Q4	“配置错了如何尽早失败？”	启动期校验
Q5	“Milvus 与 Zilliz 怎么选？”	自建/托管取舍
Q6	“Provider fallback 能否证明生产可用？”	工程诚实度

📍 代码位置详解

> **`app/core/config.py:43-100, 265-374, 504-509` — `Settings`：配置驱动的部署切换**
>
> 1. **第 43-45 行**：定义评测执行模式和 Celery Broker/Result Backend。
> 2. **第 70-100 行**：定义连接池、Redis、稀疏索引和向量库 Provider 配置。

3. **第 265-374 行**：启动期规范化并校验 `sync/background/celery`、`memory/opensearch`、`local/milvus/zilliz` 等合法组合；Celery 模式缺 Broker 会直接报错。
4. **第 504-509 行**：兼容 Redis URL 和分字段配置，统一生成连接地址。

> **`app/integrations/vector_store.py:47-...`、`app/retrieval/sparse_index.py:201-418` 与 `app/tasks/evaluation_tasks.py:18-58` — 外部基础设施接口**
>
> 向量库客户端、稀疏索引 Provider 和 Celery Task 将业务层与 Local/Milvus/Zill/OpenSearch/Redis 具体实现隔离；切换配置不应改变 `RetrievalService` 的调用契约。

🔧 技术初遇详解：Provider 策略模式

🔌 Provider — 可替换的基础设施适配层

是什么：Provider [注解：对同一能力定义稳定接口，再由本地实现或外部服务实现该接口的适配层]。它支持本地快速验证和生产基础设施之间的切换。

🎯 Q1：为什么本地内存模式不能直接扩成多实例生产？

👤 **面试官**：本地索引运行得很好，为什么多开几个 API 实例不行？

💬 **你**：每个进程的内存索引和去重缓存都互相隔离。文档在实例 A 摄取后，实例 B 还没有索引；负载均衡后同一用户的请求会命中不同实例，结果不一致。重启后所有索引丢失，需要全量重建。

更关键的是规模问题。本地 FAISS 索引在 10 万向量时没有什么压力——内存占用几百 MB，查询 < 5ms。但百万级向量时：HNSW 索引需要 8-16GB 内存，全量重建耗时数十分钟，多实例各存一份是巨大的资源浪费。本地 BM25 每次 `ensure_database_indexed()` 都会扫描全量 Chunk 表重建倒排索引——百万片段时这个操作本身就变成瓶颈。生产必须把向量存到 Milvus/Zilliz、稀疏索引切到 OpenSearch——它们持久化存储、支持增量更新、多实例共享同一个索引、重启不需要重建。

🎯 Q2-Q6：快速应答表

问题简述	一句话应答
Celery/Redis 的价值?	将长任务移出 HTTP 进程，支持 Broker 分发和 Worker 扩缩，但仍要处理幂等、重试和死信。
配置如何防错?	`Settings` 在启动期校验 Provider、执行模式和依赖配置，避免运行时静默降级。
Milvus 与 Zilliz 如何选?	自建 Milvus 便于内网和成本控制，托管 Zilliz 降低运维；都需压测和权限评估。
fallback 是否等于上线?	❌ 不等于；它只是代码级回退边界，仍需集群联调、压测和故障演练。
为什么策略模式适合这里?	调用方依赖稳定接口，替换基础设施不需要改检索/问答业务代码。

🔄 同类技术对比：运行时基础设施

维度	进程内存	自建 Milvus/OpenSearch	托管 Zilliz	本项目策略
多实例一致性	❌	✅	✅	本地/生产可切换
重启持久化	❌	✅	✅	生产不用内存模式
运维成本	✅ 最低	❌ 较高	⚠️ 中	按环境选择
数据控制	✅ 本机	✅ 私有部署	⚠️ 受云服务约束	企业按合规选型

4.2 权限、连接池、健康探针与指标体系

💡 你能被问到的所有问题（共 6 题）

#	问题	考察点
Q1	“为什么权限必须在检索/工具层强制执行？”	纵深防御
Q2	“连接池的 timeout、pre_ping、recycle 分别防什么？”	数据库资源治理
Q3	“健康检查和 Prometheus 指标的职责为何不同？”	运维体系

#	问题	考察点
Q4	“Histogram 如何用于定位 P99？”	延迟分析
Q5	“为什么不能把 user_id 打到 Prometheus 标签？”	高基数风险
Q6	“如何从用户投诉定位到一次回答？”	Trace 与日志关联

📍 代码位置详解

> **`app/permissions/loader.py:56-94` 、 `app/db/runtime.py:18-100` 、 `app/services/health.py:21-96` 与 `app/core/metrics.py:1-187` — 企业运行时治理**
>
> 1. **`loader.py:56-94`**：按 `permission_mode` 加载 NoOp 或动态权限插件；生产需启用插件，不能依赖模型 Prompt。
> 2. **`runtime.py:31-40`**：通过 `pool_pre_ping` 、 `pool_size` 、 `pool_timeout` 、 `pool_recycle` 管理数据库连接生命周期。
> 3. **`health.py:32-89`**：分别探测 PostgreSQL、Redis，并区分 `not_configured` 、 `configured` 与深度探针结果。
> 4. **`metrics.py:1-187`**：定义 HTTP、LLM、Embedding、检索、向量库指标并输出 Prometheus 格式，Grafana 负责展示和告警。

🔧 技术初遇详解：Prometheus + Grafana

🔌 可观测性 — 用指标、日志和 Trace 解释系统正在发生什么

是什么：可观测性 [注解：从系统输出的指标、日志和调用链推断内部状态并定位问题的能力]。健康探针回答“能否服务”，指标回答“长期趋势和分位数”，Trace/日志回答“这次请求发生了什么”。

🎯 Q1：为什么权限不能交给 LLM 自觉，健康检查也不能代替指标?

👤 **面试官**：Prompt 写“不要回答敏感文档”不就够了吗?

💬 **你**：不够。Prompt 是概率性软约束，权限必须在 `PermissionChecker` 和检索过滤数据通道中强制执行。即使模型被注入或生成异常，未授权文档也不应进入候选和上下文。

🎯 Q2：健康检查为什么不能替代 Prometheus 指标?

👤 **面试官**：既然有 `/health` ，为什么还要 Prometheus?

💬 **你**：健康检查是瞬时可用性判断，Prometheus 记录请求量、错误量、Token、检索/LLM 延迟分布等时间序列；Grafana 用它观察趋势、P95/P99 和告警。两者解决的问题不同，不能互相替代。

🎯 Q3-Q6：快速应答表

问题简述	一句话应答
pool_timeout/pre_ping/recycle?	分别限制等待、取用前验活、回收老连接，构成数据库资源保护。
为什么不能用 user_id 标签?	会产生高基数时序，拖垮 Prometheus 存储和查询；用户维度应放日志/Trace。
如何定位慢请求?	用 Trace 关联 API、检索日志和 LLM 日志，再以 Histogram 分位数确定瓶颈层。
configured 与 down 的差别?	前者表示配置存在，后者表示实际探针失败；可帮助区分漏配和故障。
NoOp 权限可以上线吗?	❌ 不可以；仅用于本地开发，生产必须接入身份和权限插件。

🔄 同类技术对比：运行时诊断手段

维度	健康检查	Prometheus 指标	日志/Trace
回答的问题	当前可用吗	趋势/分位数如何	单次请求发生什么
数据形式	状态快照	时序聚合	事件明细
告警能力	⚠️ 基础	✅ 主要手段	⚠️ 需额外规则
本项目位置	`health.py`	`metrics.py` + Grafana	Trace/检索/LLM 日志

📋 第4天自检清单（升级版）

基础层（必须全部过关）

- ☐ 能说明 Local、Milvus/Zilliz、OpenSearch、Redis/Celery 分别存储或处理什么状态。
- ☐ 能解释策略模式为何能支持开发态到生产态切换。
- ☐ 能区分健康检查、指标、日志和 Trace。

进阶层（模拟面试追问）

- ☐ 能回答多实例内存索引不一致、进程内去重失效和长任务阻塞问题。
- ☐ 能说明连接池保护、权限下推和 Prometheus 高基数风险。

对比层（体现技术视野）

- ☐ 能比较自建 Milvus/OpenSearch 与托管 Zilliz 的运维/合规取舍。
- ☐ 能准确区分“Provider 代码接入”与“生产集群联调完成”。

校园场景专项：第 4 天加练（30 分钟）

场景口径与数据边界

问题	推荐回答
落地范围到底多大?	教务处、学生处、计算机学院试点运行。对接学校统一认证（CAS/OAuth），用户以学号/工号登录。 从各系统公开发布的页面和文件中索引文档，不直连任何业务数据库。实际校园自有文档万级， 飞书群机器人在试点部门群运行。
校园数据来自哪里?	学校官网和信息公开网（公开制度、通知）、从教务/学工/后勤等系统公开发布的页面和文件中索引（爬取公开页面， 非直连数据库）、自有课程/实验/论文资料。
敏感数据怎么处理?	入库但受权限保护。学生成绩、未发表实验数据、内部管理制度通过五级权限模型（公开/校内/院系/角色/私有）管控， 越权文档在检索层就被过滤，不会进入用户可见候选集。
数据规模怎么说?	校园自有数据十万级原始文档，经六类策略切分后形成百万级知识片段。公开语料和扩容数据用于压力测试， 验证更大规模下的检索延迟和并发能力。
能说系统已全校推广吗?	已在校园场景实际部署运行，核心模块全链路跑通。大规模用户推广和持续运营是后续阶段—— 当前重点是验证系统工程能力和可复现的评测指标。

校园权限模型

```
public       -> 无需认证即可访问，例如公开通知、公开办事指南
internal     -> 已认证校内用户，例如通识课程资料
department   -> owner_dept 与用户学院/部门匹配，例如课题组或学院资料
role         -> owner_role 与用户角色匹配，例如研究生、教师、管理员资料
private      -> 仅创建者，例如个人工作草稿；未知级别默认拒绝
```

代码链路：网关或会话生成可信身份 -> `app/api/deps.py` 解析本地用户的角色/学院 -> `app/services/retrieval.py` 拒绝请求体伪造身份 -> `app/permissions/campus.py` 过滤文档 -> 向量/稀疏检索只处理可见候选。

性能与部署追问

追问	回答要点
如何证明百万级能力?	用"百万级知识片段"而非"百万级校园原始文档"表述；在 Milvus/OpenSearch 环境下使用 `scripts/campus_retrieval_benchmark.py` 记录 p50/p95/p99、并发 QPS、Recall@K 和 MRR。
为什么不测完整问答 P95?	LLM 生成受模型、网络和输出长度影响，必须与纯检索延迟拆开统计;"毫秒级响应"只适用于预热后的检索服务，端到端问答延迟需单独标注 LLM 生成耗时。
检索延迟随索引规模怎么变化?	HNSW 索引的查询复杂度是 O(log N)，100 万向量和 1000 万向量的延迟差在一个数量级内。 实际瓶颈通常在网络和序列化，而非索引算法本身。
并发测试怎么做的?	`campus_retrieval_benchmark.py` 支持多线程并发查询，分别记录不同并发数下的延迟分位数和 QPS，定位瓶颈在 Embedding 生成还是 Milvus 查询。
生产为什么不用本地索引?	本地向量与内存 BM25 无法在多副本间共享，重启会丢失且每次校验会扫描全量数据；生产改用 Milvus/Zilliz 与 OpenSearch，索引持久化且多实例一致。
网关注入身份头安全吗?	必须由网关剥离客户端同名头后重新注入，服务端再从身份表加载角色/学院。 `app/api/deps.py` 只信任网关注入的 `X-User-Id` ，不接受请求体中的 `roles` 字段。
用户绕过前端直接调 API 怎么办?	API 层本身不做最终鉴权—— `deps.py` 从网关 Header 解析身份后， `retrieval.py` 在检索前调用 `CampusPermissionChecker` 过滤可见文档。即使直接调 API，未认证请求只能访问 `access_level=public` 的文档。
文件夹同步重复文件怎么去重?	`source_syncs.py` 以文件相对路径哈希作为唯一键，已同步且内容未变的文件跳过； `_validate_folder_source_path` 限制同步根目录在白名单内，防止路径穿越。
如果换成企业数据，哪些要改?	数据灌入后改三处：权限插件从 `CampusPermissionChecker` 换成企业 LDAP/RBAC 实现； `source_module` 标签从"教务处/财务处"换成企业部门名；评测集从校园真题换成企业 FAQ。检索和问答层代码不改。

最终自检

- ☐ 能用"校园验证、企业化架构、未宣称生产上线"三个关键词讲清项目定位。
- ☐ 能解释 `source_module` 、 `access_level` 、 `owner_dept` 、 `owner_role` 的区别和代码位置。
- ☐ 能说明 `/sources` 批量同步的增量机制、相对路径规范化和公开数据合规边界。
- ☐ 能区分固定评测集指标（80 条 Hit@1/MRR）、压力测试指标（P95/QPS）和大规模用户行为指标。
- ☐ 能讲清检索延迟和 LLM 生成延迟为什么要分开统计。
- ☐ 能解释网关身份注入的安全模型：为什么不能信任客户端提交的 `roles` 。

附录 A：简历逐句代码映射速查表

简历关键词	最佳代码证据	行号	面试时的准确表述
解析器注册表	`app/ingestion/parser_registry.py`	18-111	多格式解析通过注册表和工厂解耦。
摄取任务	`app/services/ingestion.py`	56-261	处理、失败标记与恢复入口在服务层。
OCR/表格识别	`app/integrations/image_recognition.py`	54-341	PaddleOCR Provider 统一图片识别和表格结果。
视觉语言模型描述	`app/integrations/image_recognition.py` 、 `image_caption.py`	342-535、26-132	Qwen-VL 类实现补足图表与流程图视觉语义。
分块在线预览	`app/api/chunking.py`	29-93	支持策略列表、自由文本和已上传文档预览。
结构分块	`app/ingestion/chunking.py`	180-380	标题和正文绑定，处理跨页噪声。
表格感知	`app/ingestion/chunking.py`	381-538	表头与行内容共同进入 Chunk。
父子分层	`app/ingestion/chunking.py`	539-645	子块召回、父块补足上下文。

简历关键词	最佳代码证据	行号	面试时的准确表述
Dense 检索	app/services/retrieval.py	199-333	生成 Query Embedding 后查询向量库。
中文 BM25	app/retrieval/sparse_index.py	30-182	倒排索引配合中文二元切词。
Hybrid 融合	app/retrieval/hybrid.py	47-102	Min-Max 后按 alpha 加权。
3 倍候选	app/services/retrieval.py	334-365	Hybrid 先扩召回再融合。
权限下推	app/services/retrieval.py 、 app/permissions/campus.py	699-778、17-58	access_level / owner_dept / owner_role 三级 fail-closed 过滤，未知级别默认拒绝。
校园元数据	app/api/documents.py 、 app/services/indexing.py	159-179、71-79	access_level / owner_dept / owner_role 随文档入库并进入 Chunk 元数据。
批量资料同步	app/api/sources.py 、 app/services/source_syncs.py	27-84、38-111	文件夹/对象存储同步，含路径校验和增量摄取。
检索候选优化	app/services/retrieval.py	217-333、735	向量召回 Chunk 后通过 _load_documents_by_uuid 批量回查文档元数据，避免全表 JOIN。
Rerank 回退	app/services/retrieval.py	507-606	可选精排失败保留原排序。
80 条校园评测基准	scripts/interview_metrics_benchmark.py	全文	校园场景固定评测集指标，不代表大规模用户行为数据。
Agent 主流程	app/services/qa_agent.py	54-211	有限状态工作流，非 ReAct。
证据阈值	app/services/qa_agent.py	281-317	按策略阈值决定回答或拒答。
JSON 改写	app/services/qa_agent.py	223-280	LLM 改写有结构合约和启发式回退。
证据式回答	app/services/qa_agent.py	319-358	生成使用 Citation，Provider 失败可本地回退。
Agent 测试	tests/test_qa_agent.py	69-217	monkeypatch 验证改写和轮次分支。
评测 Run	app/services/evaluation.py	170-224	固化数据集、策略、参数和执行模式。
数据隔离	app/services/evaluation.py	404-445	复制到 evaluation_chunks 。
向量隔离	app/services/evaluation.py	446-497	使用 eval_<run_uuid> 。
Hybrid 评测	app/services/evaluation.py	498-519, 783-902	Run 专属 BM25 与临时向量集合。
后台/Celery	app/services/background_jobs.py	21-121	线程池与 Celery 均有代码实现。
飞书/企微回调	app/api/feishu.py 、 app/api/wecom.py	21-88、21-83	验签、解密、去重、异步回复。
机器人派发	app/services/bot_dispatch.py	19-108	复用问答服务并格式化 Citation 来源。
消息密码学	app/integrations/bot_crypto.py	17-151	AES-CBC、签名校验和 TTL 去重缓存。
健康检查	app/services/health.py	21-96	基础探活、配置状态和可选深度探针。
指标/权限	app/core/metrics.py 、 app/permissions/loader.py	1-187、56-94	Prometheus 指标与可插拔权限边界。
校园压测	scripts/campus_retrieval_benchmark.py	全文	输出客户端和服务端检索 p50/p95/p99，不混入 LLM 生成。

附录 B：90 秒项目自我介绍

我做的是一个面向本校校园场景的 RAG 综合知识库平台。学校本身就是一个知识密集型组织——制度、办事流程、课程培训、科研资料、多部门协作和分层权限——这些在知识管理维度上可以覆盖中型企业 80% 以上的场景需求。

离线侧，通过解析器注册表处理 PDF、Word、Excel、HTML、邮件、聊天记录和图片等十余种资料。图片侧用 PaddleOCR 提取文字和表格，用 Qwen-VL 视觉语言模型补足流程图、图表等视觉语义；再依据内容形态采用固定长度、结构化、表格感知、父子分层、语义和聊天记录六类切分策略，并提供在线预览。每个 Chunk 保存页码、章节、行范围和父块元数据，回答可回溯到原文位置。

在线检索侧，实现 Dense 语义检索与中文二元切词 BM25 的 Hybrid 双路召回，先扩召回候选（3×top_k），分别 Min-Max 归一化后按 alpha=0.7 加权融合，并支持可选 Rerank 精排及失败回退。权限通过策略模式在检索数据层下推，按公开、校内、院系、角色和私有范围做 fail-closed 过滤。

问答层不是开放式 ReAct 或多 Agent 协作，而是手写有限状态机：plan → retrieve → evidence gate → 证据不足则 LLM 改写查询重试一次 → answer → finish，最多两轮检索上界。证据门控按 Dense/Hybrid 策略使用差异化阈值，两轮仍不足则拒答而非编造。在 80 条校园场景评测集上，混合检索 Hit@1 从 58.3% 提升到 83.3%，MRR 从 0.792 提升到 0.917——这是可复现的固定评测集结果，不代表大规模用户流量指标。

生产基础设施通过策略模式支持 Local/Memory → Milvus/Zilliz/OpenSearch/Celery+Redis 平滑切换，检索业务代码不变。接入飞书/企微 Webhook（验签→解密→去重→异步回复）。监控侧有 Prometheus 15 指标 + Grafana 大盘、健康探针、数据库连接池保护及可插拔权限插件。代码层面已完整接入，但外部集群压测、备份恢复和身份联调仍需独立完成。

附录 C：4 天学习日程

天数	上午：源码主线	下午：面试输出	晚上：验证与复盘
第 1 天	`documents -> ingestion -> parser_registry -> image_recognition -> chunking`	讲清校园资料摄取、OCR/VLM、分块和引用	跑分块测试，口述制度/课程/表格的策略选择
第 2 天	`retrieval -> sparse_index -> hybrid -> qa_agent -> llm`	背 Hybrid、状态流、拒答和坏 JSON 追问	运行受控基准并修改 mock 观察 Trace
第 3 天	`permissions -> deps -> identity -> sources -> retrieval`	演练公开/校内/院系/角色权限，说明公开数据边界	跑权限、无筛选检索和资料同步定向测试
第 4 天	`evaluation -> feishu/wecom -> background_jobs -> health -> metrics -> deploy/k8s`	模拟隔离、Webhook、Celery、Provider、压测与监控追问	运行压测脚本帮助、完成 90 秒介绍和 10 分钟项目深挖

附录 D：简历功能点逐条详解（24 个核心项 + 4 个扩展项）

#	功能点	代码证据	面试要点
1	FastAPI 分层入口	`app/main.py:54-99`	路由薄、服务层承载业务。
2	多格式文档解析	`parser_registry.py:18-111`	注册表可扩展。
3	OCR/表格识别	`image_recognition.py:116-341`	PaddleOCR 提取文字与表格。
4	视觉语言模型	`image_recognition.py:342-535`	Qwen-VL 类能力理解图表/流程图。
5	摄取任务状态	`ingestion.py:56-261`	失败可定位、可恢复。
6	分块在线预览	`api/chunking.py:29-93`	支持自由文本和已上传文档预览。
7	结构分块	`chunking.py:180-380`	标题随正文。
8	表格分块	`chunking.py:381-538`	表头和行上下文。

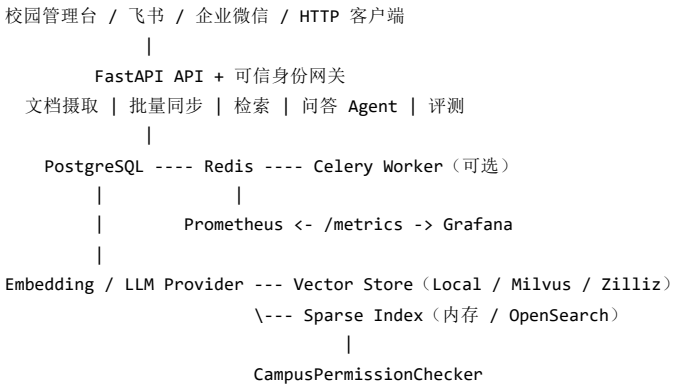
#	功能点	代码证据	面试要点
9	父子分块	`chunking.py:539-645`	小块召回、大块生成。
10	语义断点	`chunking.py:646-734`	启发式相似度，不夸大。
11	聊天记录分块	`chunking.py`	按会话消息边界组织上下文。
12	引用溯源	`models/document_chunk.py`	UUID + 定位元数据。
13	Embedding Provider	`integrations/embedding.py`	OpenAI 兼容接口与回退边界。
14	向量库 Provider	`integrations/vector_store.py`	本地/Milvus/Zilliz 可替换。
15	Dense 检索	`retrieval.py:199-333`	语义召回。
16	BM25	`sparse_index.py:30-182`	精确词召回。
17	Hybrid 融合	`hybrid.py:47-102`	归一化与 alpha。
18	Rerank	`rerank.py:28-158`	精排且可降级。
19	权限下推	`retrieval.py:699-778`	数据层强制校验。
20	Agent 状态机	`qa_agent.py:54-211`	两轮上界。
21	Query Rewrite	`qa_agent.py:223-280`	JSON + 回退。
22	证据拒答	`qa_agent.py:281-358`	降低无依据回答。
23	Agent Trace	`qa_agent.py:80-211`	可解释执行过程。
24	评测数据集/隔离	`evaluation.py:83-162,404-519`	Query、表、向量、稀疏索引快照。
25	异步评测	`background_jobs.py:21-121`	同步、线程与 Celery 三种模式。
26	IM 接入	`api/feishu.py` 、 `api/wecom.py`	验签、解密、去重、异步回复。
27	健康与连接池	`health.py:21-96` 、 `runtime.py:18-100`	探针、资源保护。
28	监控与权限插件	`metrics.py:1-187` 、 `loader.py:56-94`	可观测性和企业扩展。

附录 E：技术栈逐项详解

技术	项目中的职责	面试说法	注意边界
FastAPI	API、依赖注入、路由	用 API 层承接 HTTP 边界	不把业务堆到路由层
Pydantic	请求/响应和约束	让 top_k、轮次、状态可验证	不等于业务授权
PaddleOCR	图片文字与表格识别	适合可控的文字/版面抽取	识别质量受扫描质量影响
Qwen-VL	图表、流程图等视觉语义描述	OCR 的补充，而非替代	外部调用成本与事实校验
SQLAlchemy/Alembic	ORM、事务、迁移	关系元数据与索引 ID 关联	Session 不跨线程共享
Hybrid Retrieval	Dense + BM25 双路召回 + Min-Max 归一化 + Alpha 融合	语义与精确词互补，不是重复建设	alpha 与候选倍数是评测驱动的参数
Rerank	粗排后精排，失败回退原排序	准确率增强项，不能代替召回	可选链路， 不可用时不影响核心检索
FSM Agent	手写有限状态机：plan → retrieve → evidence gate → rewrite → answer	不是 ReAct，没用 LangGraph/LangChain，最多两轮	企业问答的受控工作流比开放 Agent 更适合
证据门控	策略差异化阈值判断证据充分性	Dense/Hybrid 分数分布不同， 不能共用阈值	阈值来自评测调参，不是理论常数
Query Rewrite	LLM 结构化 JSON 改写 + 4 级启发式回退	控制数据必须结构化， 不能依赖自由文本	坏 JSON/超时/Provider 不可用均有回退

技术	项目中的职责	面试说法	注意边界
Redis	Broker/基础设施探活/限流	支持任务队列接入和分布式限流	本地连通不等于集群验证
Milvus/Zilliz	向量检索 Provider	向量库可替换	真实外部联调待验证
OpenSearch	外部稀疏检索 Provider	稀疏索引可演进	当前需独立部署验证
Celery	长任务异步执行	Broker + Worker 支持重试	需治理幂等与可观测性
Prometheus	指标采集	Counter/Gauge/Histogram 建模	避免高基数标签（user_id、原始 query）
Grafana	指标大盘与告警展示	基于 Prometheus 做趋势和分位数可视化	不是指标采集端
React/Vite	管理台前端	文档、检索、问答、评测界面	本计划聚焦后端主线

附录 F：部署设计与拓扑详解



1. **API 层**： `app/main.py` 创建应用并注册路由； `app/api/router.py` 同时注册 `/sources` 批量同步；生产应由网关承担 TLS、认证、限流和请求体限制。
2. **关系数据层**：PostgreSQL 保存文档、Chunk、任务、评测、日志和权限关联；Alembic 管理模式迁移。
3. **检索层**：向量库保存 Embedding，BM25/OpenSearch 提供稀疏召回； `chunk_uuid/doc_uuid` 将检索记录与元数据连接；无筛选查询采用“先召回、后回查”减少全量文档扫描。
4. **异步层**：轻量场景可使用 `ThreadPoolExecutor` ；可靠长任务使用 Redis Broker + Celery Worker，需结合 Run 幂等和监控。
5. **观测层**： `/health` 用于探活， `/metrics` 供 Prometheus 拉取，Grafana 用于趋势、分位数和告警展示。
6. **持续迭代方向**：外部 Provider 压测调优、Celery 重试与死信监控、备份恢复演练、限流策略和大规模用户推广；五级权限模型和核心检索链路已在实际运行中验证。